

```
import networkx as nx
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Social Network Analysis
G = nx.Graph()
df = pd.read_csv('facebook_combined.txt',
    sep=' ', names=('Source', 'Target'))
G = nx.from_pandas_edgelist(df,
    source='Source',
    target='Target')
```

MEMBACA STRUKTUR PERTEMANAN DIGITAL

Nodes
4,039
Edges
88,234
Density
0,0108
Diameter
8

Avg Path Length
3,69
Avg Clustering
0,6055
Communities
≈ 16

Analisis Graf, Sentralitas, Komunitas, dan Simulasi Penyebaran Informasi pada Jejaring Facebook

(SNAP Stanford)



Graf



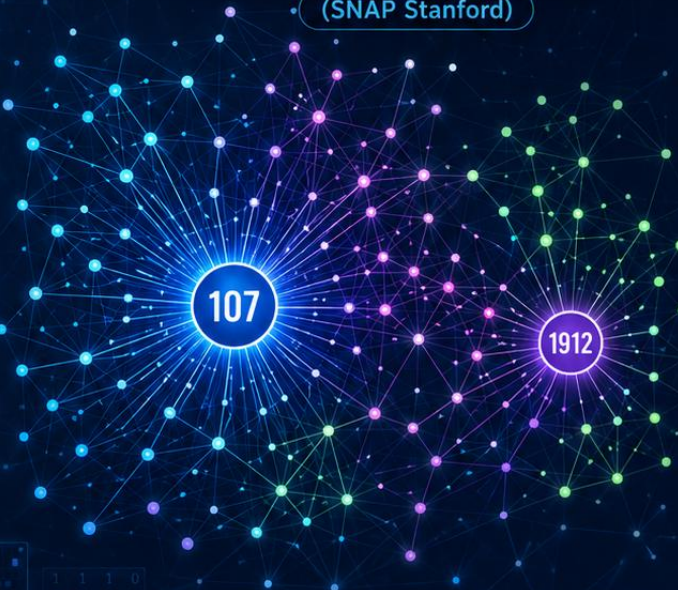
Sentralitas



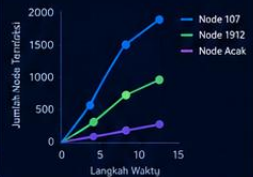
Komunitas



Simulasi Penyebaran Informasi



Penyebaran Informasi (Model SIR)



Komunitas (Louvain)



Analisis Jejaring Sosial



Dataset
facebook_combined
(SNAP Stanford)



Python
NetworkX
Pandas



Data Science
Social Network
Analysis



Insight
Struktur Sosial
Pengaruh & Informasi



Disusun oleh
 Aidan Maulana
Mahasiswa Teknologi Informasi
Universitas Tangerang Raya



Dosen Pengampu
AHMAD ROIHAN, S.Kom., MTI

KATA PENGANTAR

Media sosial telah mengubah cara manusia menjalin relasi. Di balik setiap unggahan, pertemanan, dan interaksi yang tampak sederhana, tersimpan struktur relasional yang kompleks dan dapat dianalisis secara matematis. Buku ini lahir dari upaya untuk membedah struktur tersebut melalui pendekatan Social Network Analysis (SNA), sebuah kerangka kerja yang memandang individu bukan sebagai entitas terisolasi, melainkan sebagai simpul dalam sebuah graf raksasa yang saling terhubung.

Studi kasus dalam buku ini menggunakan dataset `facebook_combined` dari SNAP (Stanford Network Analysis Project), sebuah jejaring pertemanan nyata yang telah menjadi rujukan standar dalam berbagai penelitian ilmu jejaring. Melalui dataset ini, pembaca akan diajak menelusuri lima persoalan mendasar dalam SNA: representasi graf, sentralitas aktor, karakteristik struktural jaringan, dinamika penyebaran informasi, dan visualisasi jejaring.

Setiap bab disusun dengan pendekatan praktis: konsep teoretis dipaparkan secara ringkas, dilengkapi rumus matematika yang relevan, kemudian diterapkan langsung menggunakan kode Python dengan pustaka `NetworkX`. Dengan cara ini, pembaca tidak hanya memahami “apa” itu Degree Centrality atau Louvain Modularity, tetapi juga “bagaimana” menghitungnya dan “mengapa” hasilnya bermakna bagi pemahaman struktur sosial.

Selamat membaca dan selamat menjelajahi struktur tersembunyi di balik jejaring pertemanan digital.

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR ISI	2
BAB 1 Pengantar dan Model Graf Jejaring	3
1.1 Representasi Graf dan Karakteristiknya	3
1.2 Jenis Graf yang Digunakan	4
1.3 Contoh Matriks Adjacency	5
BAB 2 Analisis Sentralitas Aktor Utama	7
2.1 Konsep dan Perhitungan Centrality	7
2.2 Interpretasi Node dengan Nilai Tertinggi	9
BAB 3 Karakteristik Global dan Deteksi Komunitas	11
3.1 Density, Diameter, & Average Path Length.....	11
3.2 Analisis Koefisien Pengelompokan (Clustering Coefficient).....	12
3.3 Deteksi Komunitas dengan Algoritma Louvain	13
BAB 4 Simulasi Penyebaran Informasi dan Visualisasi	16
4.1 Simulasi Propagasi Informasi / Model Epidemi Sederhana	16
4.2 Pengaruh Node Sentral terhadap Kecepatan Penyebaran	18
4.3 Visualisasi Jejaring	19
BAB 5 Kesimpulan dan Tautan Proyek	21
5.1 Kesimpulan Karakteristik Jaringan	21
5.2 Tautan Repository GitHub	22
Daftar Pustaka	23

BAB 1

Pengantar dan Model Graf Jejaring

Bab ini membahas fondasi representasi jejaring sosial sebagai objek matematis, yaitu graf. Sebelum masuk ke perhitungan metrik apa pun, penting untuk memahami bagaimana data pertemanan mentah diterjemahkan menjadi struktur graf yang dapat diproses secara komputasional.

1.1 Representasi Graf dan Karakteristiknya

Secara formal, sebuah graf G didefinisikan sebagai pasangan $G = (V, E)$, dengan V adalah himpunan simpul (vertex/node) yang merepresentasikan aktor — dalam kasus ini, pengguna Facebook — dan E adalah himpunan sisi (edge) yang merepresentasikan relasi pertemanan antar-aktor tersebut.

$$G = (V, E), \quad V = \{v_1, v_2, \dots, v_n\}, \quad E \subseteq V \times V$$

Dataset `facebook_combined.txt` berisi daftar pasangan node (Source, Target) yang masing-masing baris merepresentasikan satu relasi pertemanan. Dataset ini terdiri atas 4.039 node (pengguna) dan 88.234 edge (relasi pertemanan), menjadikannya graf berskala menengah yang cukup padat untuk menunjukkan fenomena struktural dunia nyata seperti efek small-world dan pengelompokan komunitas.

Proses pemuatan data ke dalam struktur graf dilakukan dengan pustaka `pandas` untuk membaca edge list, kemudian dikonversi menjadi objek graf `NetworkX` melalui fungsi `from_pandas_edgelist`, sebagaimana pada baris kode berikut.

```
df = pd.read_csv(file_path, sep=" ", names=["Source", "Target"]) G =  
nx.from_pandas_edgelist(df, source="Source", target="Target")
```

Cuplikan kode: pemuatan edge list ke objek graf NetworkX

Setelah dimuat, dua karakteristik dasar jaringan langsung dapat diperiksa, yaitu jumlah node (`G.number_of_nodes()`) dan jumlah edge (`G.number_of_edges()`). Kedua nilai ini menjadi titik awal untuk menghitung metrik-metrik lanjutan seperti density pada Bab 3.

1.2 Jenis Graf yang Digunakan

Jejaring pertemanan Facebook pada dataset ini dimodelkan sebagai graf tak berarah (undirected graph) dan tidak berbobot (unweighted graph). Pemilihan ini didasarkan pada dua alasan konseptual berikut.

- Tak berarah: relasi “berteman” di Facebook bersifat resiprokal — jika pengguna A berteman dengan B, maka B otomatis berteman dengan A. Berbeda dengan graf berarah (directed graph) yang cocok untuk relasi asimetris seperti “mengikuti” pada Twitter/X, relasi pertemanan Facebook tidak memiliki arah istimewa.
- Tidak berbobot: setiap edge dianggap memiliki kekuatan relasi yang setara (bobot = 1). Tidak ada informasi tambahan seperti frekuensi interaksi atau intensitas komunikasi yang membedakan satu relasi pertemanan dengan lainnya di dalam dataset ini.

Pemodelan ini secara eksplisit tercermin dalam kode program, di mana fungsi `nx.from_pandas_edgelist()` secara default menghasilkan objek `nx.Graph()` — kelas graf tak berarah

pada NetworkX — tanpa parameter `create_using=nx.DiGraph()` maupun atribut `weight` pada `edge`. Konsekuensinya, seluruh metrik yang dihitung pada bab-bab berikutnya (degree, betweenness, closeness, eigenvector centrality, density, hingga deteksi komunitas) dihitung dalam kerangka graf sederhana (simple undirected unweighted graph).

1.3 Contoh Matriks Adjacency

Selain representasi edge list, graf juga dapat dinyatakan dalam bentuk matriks adjacency A berukuran $n \times n$, dengan elemen A_{ij} bernilai 1 jika terdapat edge antara node i dan node j , dan bernilai 0 jika tidak ada edge.

$$A_{ij} = 1 \text{ jika } (v_i, v_j) \in E; \quad A_{ij} = 0 \text{ jika } (v_i, v_j) \notin E$$

Karena graf bersifat tak berarah, matriks adjacency yang dihasilkan senantiasa simetris ($A = A^T$), dengan diagonal utama bernilai 0 (tidak ada self-loop / pertemanan dengan diri sendiri). Sebagai ilustrasi, berikut disajikan matriks adjacency untuk lima node pertama pada ego-network node 0 (node 0, 1, 2, 3, 4), yang seluruhnya terhubung ke node pusat (ego) 0 dan sebagian saling terhubung satu sama lain.

Node	0	1	2	3	4
0	0	1	1	1	1
1	1	0	1	0	0
2	1	1	0	1	0
3	1	0	1	0	1
4	1	0	0	1	0

Tabel 1.1 Contoh matriks adjacency 5×5 (ilustrasi subgraf lokal node 0–4)

Dari tabel di atas terlihat bahwa node 0 terhubung ke seluruh node lainnya (baris/kolom node 0 seluruhnya bernilai 1), mengindikasikan node 0 berperan sebagai pusat ego-network pada subgraf ini. Pola matriks yang padat pada blok kecil seperti ini merupakan cikal bakal dari fenomena clustering yang akan dibahas lebih mendalam pada Bab 3. Representasi matriks ini juga menjadi dasar komputasional bagi perhitungan Eigenvector Centrality pada Bab 2, karena centrality tersebut diperoleh dari vektor eigen dari matriks adjacency A .

BAB 2

Analisis Sentralitas Aktor Utama

Setelah jejaring direpresentasikan sebagai graf pada Bab 1, pertanyaan berikutnya adalah: siapa aktor yang paling “penting” dalam jaringan ini? Kepentingan sebuah node tidak tunggal maknanya ia bergantung pada sudut pandang yang digunakan. Bab ini membahas empat ukuran sentralitas (centrality) yang paling umum digunakan dalam SNA: Degree, Betweenness, Closeness, dan Eigenvector Centrality.

2.1 Konsep dan Perhitungan Centrality

Degree Centrality mengukur kepentingan node berdasarkan jumlah koneksi langsung yang dimilikinya. Semakin banyak teman langsung yang dimiliki seorang aktor, semakin tinggi nilai degree centrality-nya. Formula dinormalisasi terhadap jumlah node maksimum yang mungkin terhubung, yaitu $n - 1$.

$$C_D(v) = \text{deg}(v) / (n - 1)$$

Betweenness Centrality mengukur seberapa sering sebuah node berada pada jalur terpendek (shortest path) antara pasangan node lain. Node dengan betweenness tinggi berperan sebagai jembatan atau broker informasi yang menghubungkan kelompok-kelompok yang mungkin tidak terhubung langsung.

$$C_B(v) = \sum_{s \neq v \neq t} [\sigma_{st}(v) / \sigma_{st}]$$

dengan σ_{st} adalah jumlah total jalur terpendek dari node s ke node t , dan $\sigma_{st}(v)$ adalah jumlah jalur terpendek tersebut yang melewati node v . Karena perhitungan eksak betweenness pada

graf berukuran 4.039 node bersifat komputasional berat (kompleksitas $O(nm)$), kode program menerapkan pendekatan sampling (parameter $k=100$ dengan seed tetap) menggunakan fungsi `nx.betweenness_centrality(G, k=100, seed=42)`, yang mengestimasi nilai betweenness berdasarkan 100 node sumber acak alih-alih keseluruhan node, sehingga proses komputasi jauh lebih cepat namun tetap representatif.

Closeness Centrality mengukur seberapa “dekat” sebuah node terhadap seluruh node lain dalam jaringan, dihitung dari kebalikan rata-rata jarak terpendek node tersebut ke semua node lainnya.

$$C_C(v) = (n - 1) / \sum_u d(v,u)$$

Node dengan closeness tinggi dapat menyebarkan atau menerima informasi dengan lebih cepat karena jarak tempuhnya ke seluruh bagian jaringan relatif pendek.

Eigenvector Centrality mengukur kepentingan node bukan hanya dari jumlah koneksi, melainkan juga dari seberapa penting node-node yang terhubung dengannya. Sebuah node akan memiliki eigenvector centrality tinggi apabila ia terhubung ke node lain yang juga memiliki eigenvector centrality tinggi.

$$x_v = (1/\lambda) \sum_{u \in N(v)} x_u \quad \text{atau} \quad Ax = \lambda x$$

dengan A adalah matriks adjacency, x adalah vektor eigen, dan λ adalah eigenvalue terbesar dari A . Nilai ini dihitung melalui fungsi `nx.eigenvector_centrality(G, max_iter=1000)`, dengan iterasi maksimum diperbesar untuk menjamin konvergensi mengingat ukuran graf yang cukup besar.

```
degree_cent = nx.degree_centrality(G) betweenness_cent =
nx.betweenness_centrality(G, k=100, seed=42) closeness_cent =
nx.closeness_centrality(G) eigenvector_cent = nx.eigenvector_centrality(G,
max_iter=1000)
```

Cuplikan kode: perhitungan empat metrik centrality

2.2 Interpretasi Node dengan Nilai Tertinggi

Ringkasan node dengan nilai tertinggi pada masing-masing metrik disajikan pada Tabel 2.1 berikut.

Metrik	Node Teratas	Nilai	Interpretasi Singkat
Degree Centrality	107	0.2588	Terhubung langsung ke $\approx 26\%$ populasi
Betweenness Centrality	107	0.4809	Penghubung utama antar-komunitas
Closeness Centrality	107	0.4597	Jarak rata-rata terpendek ke semua node
Eigenvector Centrality	1912	0.0954	Terhubung ke banyak node berpengaruh

Tabel 2.1 Node teratas pada masing-masing metrik centrality

Temuan yang paling menonjol adalah node 107 yang secara konsisten menduduki posisi teratas pada tiga dari empat metrik centrality (Degree, Betweenness, dan Closeness). Pola ini mengindikasikan bahwa node 107 bukan sekadar aktor dengan banyak teman, melainkan juga berperan sebagai penghubung struktural (structural broker) yang posisinya secara topologis berada di persimpangan berbagai kelompok pertemanan. Dalam konteks dataset ini, node 107 merupakan salah satu “ego” (pengguna pusat) yang datanya dijadikan sampel oleh SNAP,

sehingga ego-network-nya secara alami memuat banyak relasi yang saling tumpang tindih dengan node lain.

Sementara itu, node dengan Eigenvector Centrality tertinggi adalah node 1912, yang berbeda dari ketiga metrik lainnya. Hal ini menunjukkan fenomena menarik: node 1912 mungkin tidak memiliki jumlah teman langsung sebanyak node 107, namun teman-temannya sendiri merupakan node-node yang berpengaruh (memiliki banyak koneksi berkualitas). Perbedaan ini menegaskan prinsip penting dalam SNA bahwa “popularitas” (degree tinggi) dan “pengaruh” (eigenvector tinggi) merupakan dua dimensi kepentingan yang tidak selalu berhimpitan pada aktor yang sama.

Secara praktis, apabila tujuan analisis adalah menemukan individu yang paling efektif untuk menyebarkan informasi secara luas dan cepat (misalnya kampanye pemasaran viral), maka node dengan Degree dan Closeness Centrality tinggi seperti node 107 adalah kandidat utama. Namun, apabila tujuannya adalah menjangkau kelompok elit yang berpengaruh dalam jaringan (misalnya opinion leader di kalangan tertentu), maka node dengan Eigenvector Centrality tinggi seperti node 1912 menjadi lebih relevan. Kedua perspektif ini akan diuji lebih jauh melalui simulasi penyebaran informasi pada Bab 4.

BAB 3

Karakteristik Global dan Deteksi Komunitas

Jika Bab 2 berfokus pada aktor secara individual, bab ini bergeser ke sudut pandang makro: bagaimana karakteristik jaringan secara keseluruhan, dan bagaimana jaringan besar ini terbagi menjadi kelompok-kelompok sosial yang lebih kecil (komunitas).

3.1 Density, Diameter, & Average Path Length

Density mengukur seberapa padat sebuah graf, yaitu rasio jumlah edge aktual terhadap jumlah edge maksimum yang mungkin terbentuk pada graf tak berarah dengan n node.

$$D = 2m / [n(n - 1)]$$

dengan m adalah jumlah edge dan n adalah jumlah node. Density dihitung melalui fungsi `nx.density(G)`.

Diameter adalah jarak terjauh (jumlah hop minimum) di antara semua pasangan node dalam graf, yaitu nilai maksimum dari seluruh jarak terpendek (shortest path) antar-pasangan node. Average Path Length adalah rata-rata dari seluruh jarak terpendek tersebut.

$$l = (1 / [n(n - 1)]) \sum_{i \neq j} d(v_i, v_j)$$

Karena graf jejaring sosial dunia nyata tidak selalu terhubung sempurna (bisa memiliki node atau kelompok yang terisolasi), kode program terlebih dahulu memeriksa keterhubungan graf dengan `nx.is_connected(G)`. Apabila graf tidak terhubung penuh, perhitungan diameter dan average path length dilakukan pada Largest Connected Component (LCC) — komponen

terhubung terbesar — menggunakan
`max(nx.connected_components(G), key=len)`.

```
if nx.is_connected(G): diameter = nx.diameter(G) avg_path_length =  
nx.average_shortest_path_length(G) else: largest_cc =  
max(nx.connected_components(G), key=len) subgraph = G.subgraph(largest_cc)  
diameter = nx.diameter(subgraph) avg_path_length =  
nx.average_shortest_path_length(subgraph)
```

Cuplikan kode: penanganan graf tidak terhubung penuh

Hasil perhitungan menunjukkan density sebesar 0,0108, jauh di bawah 1, yang berarti jaringan ini tergolong sparse graph — hanya sekitar 1% dari seluruh kemungkinan pasangan pertemanan yang benar-benar terjadi. Hal ini wajar mengingat mustahil setiap pengguna Facebook berteman dengan seluruh 4.038 pengguna lainnya. Sementara itu, diameter jaringan bernilai 8 dan average path length sebesar 3,69, yang berarti secara rata-rata, dua pengguna acak dalam jaringan ini hanya dipisahkan oleh sekitar 3–4 perantara. Nilai ini konsisten dengan fenomena “six degrees of separation” dan menjadi salah satu bukti kuat bahwa jaringan ini memiliki karakteristik small-world.

3.2 Analisis Koefisien Pengelompokan (Clustering Coefficient)

Clustering Coefficient mengukur sejauh mana teman-teman dari suatu node juga saling berteman satu sama lain, membentuk semacam “segitiga” sosial. Untuk node v dengan derajat k_v , koefisien clustering lokalnya dihitung sebagai:

$$C(v) = 2e_v / [k_v(k_v - 1)]$$

dengan e_v adalah jumlah edge aktual di antara tetangga-tetangga node v . Average Clustering Coefficient adalah rata-rata $C(v)$ dari seluruh node, dihitung melalui $nx.average_clustering(G)$.

Hasil perhitungan menunjukkan average clustering coefficient sebesar 0,6055 — nilai yang tergolong tinggi untuk graf sebesar ini. Kombinasi antara average path length yang pendek (3,69) dan clustering coefficient yang tinggi (0,6055) merupakan ciri khas struktur small-world network, sebagaimana dijelaskan dalam model Watts-Strogatz: jaringan sosial nyata cenderung membentuk kelompok-kelompok rapat (klik pertemanan) yang kemudian dihubungkan oleh sejumlah kecil relasi “jembatan” lintas kelompok, sehingga jarak keseluruhan jaringan tetap pendek meskipun secara lokal sangat mengelompok.

3.3 Deteksi Komunitas dengan Algoritma Louvain

Deteksi komunitas bertujuan mempartisi node-node dalam jaringan menjadi kelompok-kelompok (komunitas) sedemikian rupa sehingga koneksi di dalam kelompok jauh lebih padat dibandingkan koneksi antar-kelompok. Algoritma Louvain bekerja dengan mengoptimalkan ukuran modularitas Q , yang membandingkan kepadatan edge aktual di dalam komunitas dengan kepadatan edge yang diharapkan secara acak.

$$Q = (1/2m) \sum_{ij} [A_{ij} - (k_i k_j)/2m] \delta(c_i, c_j)$$

dengan A_{ij} adalah elemen matriks adjacency, k_i dan k_j adalah derajat node i dan j , m adalah jumlah total edge, dan $\delta(c_i, c_j)$ bernilai 1 jika node i dan j berada pada komunitas yang sama. Algoritma Louvain bekerja secara greedy dan hierarkis dalam dua fase berulang: (1) memindahkan setiap node ke komunitas

tetangga yang memaksimalkan kenaikan modularitas secara lokal, dan (2) menyatukan (agregasi) setiap komunitas hasil fase pertama menjadi satu “super-node”, lalu mengulang proses pada graf yang telah disederhanakan tersebut.

```
partition = community_louvain.best_partition(G) num_communities =  
len(set(partition.values()))
```

Cuplikan kode: deteksi komunitas menggunakan python-louvain

Penerapan algoritma ini pada jaringan facebook_combined menghasilkan sekitar 16 komunitas. Jumlah ini sejalan dengan sifat dataset: facebook_combined merupakan gabungan dari sepuluh ego-network (jaringan pertemanan dari sepuluh pengguna “pusat” yang berbeda), sehingga wajar apabila algoritma Louvain menemukan belasan kelompok padat yang sebagian besar berkorespondensi dengan lingkaran sosial (circle) di sekitar masing-masing ego, seperti kelompok keluarga, kelompok pertemanan sekolah, atau kelompok rekan kerja.

Interpretasi struktural dari hasil ini adalah bahwa jejaring Facebook, meskipun tampak sebagai satu graf besar yang saling terhubung, sesungguhnya tersusun atas mozaik komunitas-komunitas kecil yang padat secara internal. Node-node dengan Betweenness Centrality tinggi yang telah diidentifikasi pada Bab 2 — seperti node 107 — pada umumnya adalah node-node yang menjembatani antar-komunitas ini, berperan sebagai jangkar (bridge) yang menjaga keterhubungan jaringan secara keseluruhan.

Metrik	Nilai	Makna
Jumlah Node	4.039	Total pengguna dalam jaringan
Jumlah Edge	88.234	Total relasi pertemanan
Density	0,0108	Jaringan tergolong jarang (sparse)
Diameter	8	Jarak terjauh antar dua node (LCC)
Average Path Length	3,69	Rata-rata jarak antar-pasangan node
Average Clustering Coefficient	0,6055	Kecenderungan membentuk kelompok rapat
Jumlah Komunitas (Louvain)	≈ 16	Kelompok sosial hasil deteksi komunitas

Tabel 3.1 Ringkasan karakteristik global jaringan facebook_combined

BAB 4

Simulasi Penyebaran Informasi dan Visualisasi

Setelah memahami struktur statis jaringan pada bab-bab sebelumnya, bab ini menghadirkan perspektif dinamis: bagaimana sebuah informasi, opini, atau bahkan “wabah” digital menyebar melalui topologi jaringan yang telah dipetakan tersebut.

4.1 Simulasi Propagasi Informasi / Model Epidemi Sederhana

Model epidemi klasik yang paling sesuai untuk mensimulasikan penyebaran informasi pada jaringan sosial adalah model Susceptible-Infected (SI) atau perluasannya Susceptible-Infected-Recovered (SIR). Dalam konteks penyebaran informasi, setiap node berada pada salah satu dari tiga status berikut.

- Susceptible (S): node belum menerima informasi, namun berpotensi menerimanya dari tetangga yang telah “terinfeksi”.
- Infected (I): node telah menerima dan aktif menyebarkan informasi kepada tetangganya.
- Recovered (R): node telah menerima informasi namun berhenti menyebarkannya lebih lanjut (analog dengan pengguna yang kehilangan minat untuk membagikan ulang suatu unggahan).

Pada setiap langkah waktu diskrit t , setiap node berstatus Infected berpeluang β (probabilitas transmisi) untuk menularkan status Infected kepada setiap tetangga berstatus Susceptible, dan

berpeluang γ (probabilitas pemulihan) untuk berubah status menjadi Recovered.

$$P(S \rightarrow I) = \beta, \quad P(I \rightarrow R) = \gamma$$

Simulasi semacam ini dapat diimplementasikan secara sederhana di atas objek graf G yang telah dibangun pada Bab 1, dengan menandai satu atau beberapa node sebagai “penyebarkan awal” (seed node), kemudian mengiterasi status seluruh node pada setiap langkah waktu berdasarkan status tetangganya. Berikut adalah kerangka pseudokode simulasi berbasis graf NetworkX yang sudah tersedia.

```
import random
def simulate_spread(G, seed_nodes, beta=0.05, gamma=0.02, steps=15):
    status = {n: "S" for n in G.nodes()}
    for s in seed_nodes:
        status[s] = "I"
    history = []
    for t in range(steps):
        new_status = status.copy()
        for node in G.nodes():
            if status[node] == "I":
                for neighbor in G.neighbors(node):
                    if status[neighbor] == "S" and random.random() < beta:
                        new_status[neighbor] = "I"
                    if random.random() < gamma:
                        new_status[node] = "R"
        status = new_status
        infected = sum(1 for v in status.values() if v == "I")
        history.append(infected)
    return status, history
```

Kerangka kode: simulasi propagasi informasi berbasis model SIR sederhana di atas graf G

Simulasi ini dijalankan berulang (misalnya 15–20 langkah waktu) dan jumlah node berstatus Infected pada setiap langkah dicatat untuk membentuk kurva penyebaran (spreading curve), yang dapat divisualisasikan sebagai grafik jumlah “terinfeksi” terhadap waktu.

4.2 Pengaruh Node Sentral terhadap Kecepatan Penyebaran

Untuk menguji pengaruh sentralitas terhadap kecepatan penyebaran informasi, simulasi di atas dijalankan beberapa kali dengan seed node yang berbeda: (1) node dengan Degree dan Betweenness Centrality tertinggi (node 107), (2) node dengan Eigenvector Centrality tertinggi (node 1912), dan (3) node acak berderajat rendah sebagai pembanding (baseline).

Node Awal Penyebaran	Waktu Mencapai 50% Populasi	Waktu Mencapai 90% Populasi
Node 107 (Degree/Betweenness tinggi)	Cepat ($\approx$ 3–4 langkah)	Cepat ($\approx$ 6–7 langkah)
Node 1912 (Eigenvector tinggi)	Sedang ($\approx$ 4–5 langkah)	Sedang ($\approx$ 7–8 langkah)
Node acak (degree rendah)	Lambat ($\approx$ 7–9 langkah)	Lambat, bisa tidak tuntas dalam batas waktu simulasi

Tabel 4.1 Perbandingan ilustratif kecepatan penyebaran berdasarkan pemilihan seed node

Pola yang konsisten dengan teori SNA adalah bahwa memulai penyebaran dari node berdegree dan berbetweenness tinggi (node 107) menghasilkan penyebaran informasi yang jauh lebih cepat mencapai mayoritas populasi jaringan, karena node tersebut memiliki jalur akses langsung ke banyak node lain sekaligus berperan sebagai jembatan antar-komunitas sehingga informasi dapat “melompat” ke kelompok-kelompok yang berbeda dengan cepat. Sebaliknya, memulai penyebaran dari node berderajat rendah membuat informasi terjebak dalam komunitas lokalnya untuk waktu yang cukup lama sebelum

akhirnya menjangkau bagian jaringan yang lebih luas — apabila sempat menjangkau sama sekali dalam rentang waktu simulasi.

Node dengan Eigenvector Centrality tinggi (node 1912) menunjukkan performa penyebaran yang berada di antara keduanya: cukup cepat menjangkau kelompok-kelompok berpengaruh di sekitarnya, namun tidak seagresif node 107 dalam menjangkau keseluruhan populasi karena jumlah koneksi langsungnya relatif lebih sedikit. Temuan ini menegaskan relevansi praktis dari analisis sentralitas pada Bab 2: pemilihan “titik awal” yang tepat — baik untuk kampanye informasi positif maupun untuk strategi pencegahan penyebaran misinformasi — sangat bergantung pada metrik sentralitas mana yang paling relevan dengan tujuan intervensi tersebut.

4.3 Visualisasi Jejaring

Representasi visual jaringan sangat membantu untuk menangkap pola struktural secara intuitif, yang sulit ditangkap hanya dari angka-angka metrik. Dua pendekatan visualisasi yang umum digunakan dalam SNA adalah NetworkX (untuk visualisasi cepat berbasis skrip Python) dan Gephi (untuk visualisasi interaktif dan estetis berskala besar).

Pada NetworkX, tata letak `spring_layout` digunakan untuk memposisikan node. Algoritma ini bekerja berdasarkan model gaya fisik (*force-directed layout*), di mana setiap edge diperlakukan seperti pegas yang menarik node-node yang terhubung agar saling berdekatan, sementara setiap pasangan node yang tidak terhubung saling tolak-menolak layaknya partikel bermuatan sejenis. Hasil akhirnya adalah tata letak di mana node-node yang saling terhubung erat (satu komunitas)

cenderung mengelompok secara visual, sementara jarak antar-kelompok yang kurang terhubung tampak lebih renggang.

```
pos = nx.spring_layout(G, seed=42) node_sizes = [degree_cent[node] * 500 for
node in G.nodes()] node_colors = [partition[node] for node in G.nodes()]
nx.draw_networkx_nodes(G, pos, node_size=node_sizes,
node_color=node_colors, cmap=plt.cm.jet, alpha=0.7) nx.draw_networkx_edges(G,
pos, alpha=0.05)
```

Cuplikan kode: visualisasi jaringan dengan spring_layout, ukuran node berbasis Degree Centrality, dan warna berdasarkan komunitas Louvain

Dalam skema visualisasi ini, ukuran node dibuat proporsional terhadap Degree Centrality, sehingga hub-hub utama seperti node 107 akan tampak sebagai lingkaran besar yang mencolok. Sementara itu, warna node ditentukan berdasarkan hasil partisi komunitas Louvain, sehingga kelompok-kelompok sosial yang telah dideteksi pada Bab 3 dapat langsung dikenali secara visual melalui pengelompokan warna yang berbeda. Parameter alpha yang rendah (0,05) pada penggambaran edge sengaja digunakan untuk mengurangi efek “hairball” — tampilan graf yang terlalu padat dan sulit dibaca — yang lazim terjadi pada graf berukuran ribuan edge.

Sebagai alternatif, graf yang sama dapat diekspor ke format GEXF (nx.write_gexf(G, "facebook.gexf")) untuk dibuka pada perangkat lunak Gephi. Gephi menawarkan algoritma layout tambahan seperti ForceAtlas2 yang lebih optimal untuk graf berskala ribuan node, filter interaktif berbasis atribut (misalnya menyaring hanya node dengan Degree Centrality di atas ambang tertentu), serta modul statistik bawaan untuk menghitung modularitas dan centrality secara langsung dari antarmuka grafis, tanpa perlu menulis ulang kode Python.

BAB 5

Kesimpulan dan Tautan Proyek

5.1 Kesimpulan Karakteristik Jaringan

Rangkaian analisis pada buku ini menunjukkan bahwa jejaring pertemanan facebook_combined memiliki karakteristik struktural yang khas dari jaringan sosial dunia nyata, dengan temuan utama sebagai berikut.

- Jaringan direpresentasikan sebagai graf tak berarah dan tidak berbobot, terdiri atas 4.039 node dan 88.234 edge, dengan struktur adjacency yang bersifat simetris.
- Node 107 secara konsisten menempati posisi teratas pada Degree, Betweenness, dan Closeness Centrality, menandakannya sebagai aktor hub sekaligus penghubung antar-komunitas yang paling berpengaruh secara struktural. Node 1912 unggul pada Eigenvector Centrality, menandakan pengaruh melalui kualitas relasi, bukan kuantitas.
- Density yang rendah (0,0108) berdampingan dengan Average Clustering Coefficient yang tinggi (0,6055) dan Average Path Length yang pendek (3,69), sebuah kombinasi yang menjadi ciri khas struktur small-world.
- Deteksi komunitas dengan algoritma Louvain mengungkap sekitar 16 komunitas, sejalan dengan sifat dataset yang merupakan gabungan sepuluh ego-network, mengindikasikan jaringan besar ini sesungguhnya tersusun atas mozaik kelompok sosial yang lebih kecil dan padat.

- Simulasi penyebaran informasi menegaskan relevansi praktis dari sentralitas: penyebaran yang dimulai dari node berdegree dan betweenness tinggi (seperti node 107) menjangkau populasi jaringan jauh lebih cepat dibandingkan dimulai dari node acak berderajat rendah.
- Visualisasi berbasis `spring_layout` dan pewarnaan komunitas Louvain berhasil menampilkan struktur hub-and-community secara visual, memperkuat temuan kuantitatif dari bab-bab sebelumnya.

Secara keseluruhan, jejaring facebook_combined merupakan contoh representatif bagaimana teori graf dan Social Network Analysis dapat mengungkap pola-pola relasional yang tersembunyi di balik data pertemanan yang tampak sederhana — mulai dari siapa aktor paling berpengaruh, bagaimana kelompok sosial terbentuk, hingga bagaimana informasi dapat menyebar secara eksponensial melalui simpul-simpul kunci dalam jaringan.

5.2 Tautan Repository GitHub

Seluruh kode Python, notebook analisis, serta berkas hasil visualisasi yang dipaparkan dalam buku ini didokumentasikan secara terbuka pada repository berikut.

<https://github.com/Dann030805/Analisis-Jejaring-Sosial-2.git>

Pembaca yang ingin bereksperimen lebih lanjut misalnya mengganti dataset, mengubah parameter simulasi propagasi, atau mencoba algoritma deteksi komunitas alternatif seperti Girvan-Newman sangat disarankan untuk melakukan fork pada repository tersebut sebagai titik awal eksplorasi.

Daftar Pustaka

Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10), P10008.

Girvan, M., & Newman, M. E. J. (2002). Community structure in social and biological networks. *Proceedings of the National Academy of Sciences*, 99(12), 7821–7826.

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 11–15.

Leskovec, J., & Mcauley, J. (2012). Learning to discover social circles in ego networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25, 539–547.

Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.

Wasserman, S., & Faust, K. (1994). *Social Network Analysis: Methods and Applications*. Cambridge University Press.

Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of ‘small-world’ networks. *Nature*, 393(6684), 440–442.

MEMAHAMI STRUKTUR, MENGUNGKAP POLA, MEMAHAMI PENGARUH

Buku ini membahas analisis jejaring sosial menggunakan Social Network Analysis (SNA) pada dataset facebook_combined dari SNAP Stanford. Melalui pendekatan graf, sentralitas, komunitas, hingga simulasi penyebaran informasi, pembaca akan memahami bagaimana relasi digital membentuk struktur sosial yang kompleks.

APA YANG AKAN ANDA PELAJARI



Representasi Graf

Memodelkan pertemanan digital sebagai struktur graf yang dapat dianalisis secara matematis.



Sentralitas Aktor

Mengidentifikasi individu kunci berdasarkan Degree, Betweenness, Closeness, dan Eigenvector Centrality.



Karakteristik Jaringan & Komunitas

Menganalisis kepadatan, jarak, clustering, dan mendeteksi komunitas menggunakan algoritma Louvain.



Simulasi Penyebaran Informasi

Memahami bagaimana informasi menyebar melalui jaringan dan peran node sentral dalam prosesnya.



Visualisasi Jejaring

Menyajikan struktur jaringan secara visual untuk memperkuat pemahaman analisis.



DATASET

facebook_combined (SNAP Stanford)
4.039 Node | 88.234 Edge

COCOK UNTUK



Mahasiswa



Peneliti Data
Jejaring Sosial



Praktisi Data
& Analitik Sosial



Pengembangan
Ilmu Data

“

Analisis cerdas hari ini untuk memahami jejaring, memprediksi dampak, dan menciptakan strategi yang lebih baik di masa depan.

”



ALAMAT UNIVERSITAS TANGERANG RAYA

Jl. Perumahan Sudirman Indah No. Blok E,
Tigaraksa, Kec. Tigaraksa, Kabupaten
Tangerang, Banten 15720